

Disjoint set (Union-Find)

You are given n elements numbered from **1** to **n** . Initially, each element is in its own group and its group's representative as well. You need to process **k** queries of two types:

- Type 1: **1 x z**
Performs a **UNION** operation on elements **x** and **z** . After merging, the representative of **z** becomes the representative of the whole group. Merge the group containing **x** into the group containing **z** by making the representative of **x** point to the ultimate representative of **z** .
- Type 2: **2 x**
Perform a **FIND** operation on element **x** . Output the representative of the group containing **x** .

Note:

- The representative of an element is the root of its group, i.e., the element whose parent is itself.
- Initially, every element is its own representative.

Examples:

Input: $n = 5$, $k = 4$, queries[] = [[2, 4], [2, 1], [1, 3, 1], [2, 3]]

Output: [4, 1, 1]

Explanation:

2 4 → Find the representative of 4. Since 4 is initially in its own group, the answer is 4.

2 1 → Find the representative of 1. Since 1 is initially in its own group, the answer is 1.

1 3 1 → Union the groups containing 3 and 1. The representative of 3 is attached to the representative of 1. So the parent of 3 becomes 1.

2 3 → Find the representative of 3. Now 3 belongs to the group whose leader is 1, so the answer is 1.

Input: $n = 6$, $k = 6$, queries[] = [[1, 2, 3], [1, 4, 5], [2, 2], [2, 3], [1, 3, 5], [2, 4]]

Output: [3, 3, 5]

Explanation:

1 2 3 → Merge group of 2 into group of 3. Now 2 points to 3.

1 4 5 → Merge group of 4 into group of 5. Now 4 points to 5.

2 2 → Representative of 2 is 3.

2 3 → Representative of 3 is 3.

1 3 5 → Merge group of 3 into group of 5. Now 3 points to 5, and 2 also indirectly belongs to the same group.

2 4 → Representative of 4 is 5.

Constraints:

$1 \leq n, k \leq 1000$

$1 \leq x, z \leq n$

```
class Solution {  
  
    public:  
  
        vector<int> DSU(int n, vector<vector<int>>& queries) {  
  
            // code here  
  
        }  
};
```

Euler circuit and Path

An Eulerian Path is a path in graph that visits every edge exactly once. An Eulerian Circuit is an Eulerian Path which starts and ends on the same vertex. Given an undirected graph with **V** nodes, and **E** edges, with adjacency list **adj**, where **adj[i]** stores all the nodes that have an edge with i, return 2 if the graph contains an eulerian circuit, else if the graph contains an eulerian path, return 1, otherwise, return 0.

Examples

Input:

Output: 2

Explanation:

Following is an eulerian circuit in the mentioned graph

1 -> 2 -> 0 -> 1

Input:

Output: 1

Explanation:

Following is an eulerian path in the mentioned graph

1 -> 0 -> 2

Constraints:

$1 \leq V, E \leq 10^4$

$1 \leq \text{adj}[i][j] \leq V-1$

```
class Solution {
```

```
public:
```

```
    int isEulerCircuit(int V, vector<vector<int>>& adj) {
```

```
// code here

}

};
```

Find The Town Judge

In a town, there are n people labeled from 1 to n . There is a rumor that one of these people is secretly the town judge.

If the town judge exists, then:

1. The town judge trusts nobody.
2. Everybody (except for the town judge) trusts the town judge.
3. There is exactly one person that satisfies properties **1** and **2**.

You are given an array `trust` where `trust[i] = [ai, bi]` representing that the person labeled a_i trusts the person labeled b_i . If a trust relationship does not exist in trust array, then such a trust relationship does not exist.

Return *the label of the town judge if the town judge exists and can be identified, or return -1 otherwise.*

Example 1:

Input: $n = 2$, `trust = [[1,2]]`

Output: 2

Example 2:

Input: $n = 3$, `trust = [[1,3],[2,3]]`

Output: 3

Example 3:

Input: $n = 3$, `trust = [[1,3],[2,3],[3,1]]`

Output: -1

Constraints:

- $1 \leq n \leq 1000$
- $0 \leq \text{trust.length} \leq 10^4$
- $\text{trust}[i].\text{length} == 2$
- All the pairs of trust are **unique**.
- $a_i \neq b_i$
- $1 \leq a_i, b_i \leq n$

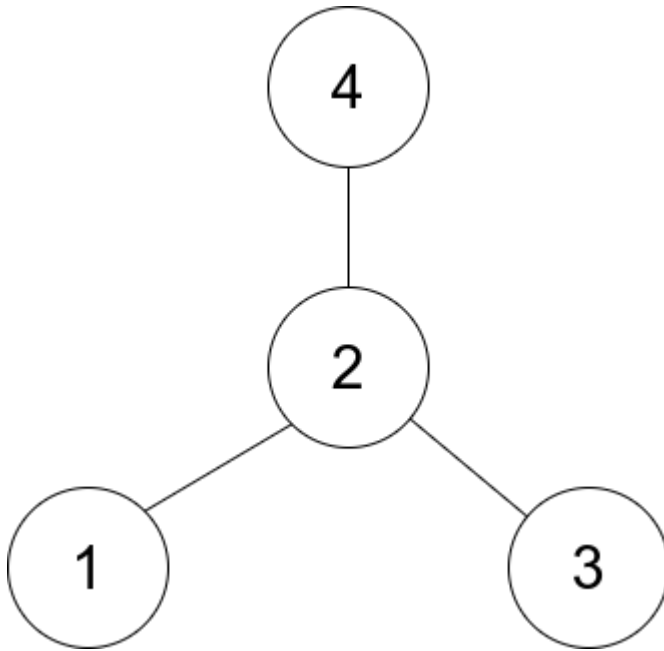
```
• class Solution {  
• public:  
•     int findJudge(int n, vector<vector<int>>& trust) {  
•  
•     }  
• };
```

Find Center of Star Graph

There is an undirected star graph consisting of n nodes labeled from 1 to n . A star graph is a graph where there is one center node and exactly $n - 1$ edges that connect the center node with every other node.

You are given a 2D integer array `edges` where each `edges[i] = [ui, vi]` indicates that there is an edge between the nodes u_i and v_i . Return the center of the given star graph.

Example 1:



Input: edges = [[1,2],[2,3],[4,2]]

Output: 2

Explanation: As shown in the figure above, node 2 is connected to every other node, so 2 is the center.

Example 2:

Input: edges = [[1,2],[5,1],[1,3],[1,4]]

Output: 1

Constraints:

- $3 \leq n \leq 10^5$
- `edges.length == n - 1`
- `edges[i].length == 2`
- $1 \leq u_i, v_i \leq n$
- $u_i \neq v_i$
- The given edges represent a valid star graph.

```

class Solution {
public:
    int findCenter(vector<vector<int>>& edges) {

    }
};

```

Path with good node

Problem Description

Given a tree with N nodes labelled from 1 to N.

Each node is either good or bad denoted by binary array A of size N where if A[i] is 1 then ith node is good else if A[i] is 0 then ith node is bad.

Also the given tree is rooted at node 1 and you need to tell the number of root to leaf paths in the tree that contain not more than C good nodes.

NOTE:

- Each edge in the tree is bi-directional.

Problem Constraints

$2 \leq N \leq 10^5$

A[i] = 0 or A[i] = 1

$0 \leq C \leq N$

Input Format

First argument is an binary integer array A of size N.

Second argument is a 2-D array B of size (N-1) x 2 denoting the edge of the tree.

Third argument is an integer C.

Output Format

Return an integer denoting the number of root to leaf paths in the tree that contain not more than C good nodes.

Example Input

Input 1:

A = [0, 1, 0, 1, 1, 1]

B = [[1, 2]

 [1, 5]

 [1, 6]

 [2, 3]

 [2, 4]

]

C = 1

Example Output

Output 1:

3

Example Explanation

Explanation 1:

Path (1 - 2 - 3) and (1 - 5) and (1 - 6) are the paths which contain atmost C nodes.

```
int Solution::solve(vector<int> &A, vector<vector<int> > &B, int C) {
```

}